

Frontend Technical Documentation

Complete File-by-File, Line-by-Line Code Breakdown & Detailed Explanations

Overview: This document provides an in-depth breakdown of every file in the frontend application (**Cust UI**) for the Customer Management System. Built with **React 19, Vite, and Axios**, the application implements a decoupled Single Page Application (SPA) architecture communicating with a Spring Boot REST API at <http://localhost:9991>. Below, each file is presented with its complete source code (excluding imports), a line-by-line breakdown of every variable and function, and a detailed explanatory paragraph.

1. File: CustomerService.js

Path: src/services/CustomerService.js • **Role:** API Communication Layer (Axios)

```
const API_URL = "http://localhost:9991";

// GET ALL CUSTOMERS
const getAllCustomers = () => {
  return axios.get(`${API_URL}/getCustList`);
};

// GET ONE CUSTOMER
const getCustomerById = (cid) => {
  return axios.get(`${API_URL}/getCust/${cid}`);
};

// CREATE CUSTOMER
const createCustomer = (customer) => {
  return axios.post(`${API_URL}/createCust`, customer);
};

// UPDATE CUSTOMER
const updateCustomer = (cid, customer) => {
  return axios.put(`${API_URL}/updateCust/${cid}`, customer);
};

// DELETE CUSTOMER
const deleteCustomer = (cid) => {
  return axios.delete(`${API_URL}/delCust/${cid}`);
};

export default {
  getAllCustomers,
  getCustomerById,
  createCustomer,
  updateCustomer,
  deleteCustomer
};
```

Line-by-Line Breakdown:

- **const API_URL = "http://localhost:9991";** — Defines the base URL pointing to the Spring Boot backend running on port 9991 so endpoints are never hardcoded across components.
- **getAllCustomers():** Sends an HTTP GET request to /getCustList. Returns a Promise resolving to the list of all customers from MySQL.

- **getCustomerById(cid)**: Sends an HTTP GET request appending the customer ID in the path (e.g. /getCust/5) to fetch a single record.
- **createCustomer(customer)**: Sends an HTTP POST request to /createCust carrying the customer object in the request body as JSON for Spring Boot to save.
- **updateCustomer(cid, customer)**: Sends an HTTP PUT request to /updateCust/{cid} with updated attributes to replace existing database values.
- **deleteCustomer(cid)**: Sends an HTTP DELETE request to /delCust/{cid} with the customer ID to delete that record from MySQL.
- **export default { ... }**: Exports all five functions bundled inside a default object so any component can invoke them cleanly.

In-Depth Explanation — CustomerService.js

This file implements the **Service Layer pattern** in frontend development. In a well-structured application, UI components should only handle rendering visual elements and managing user interactions—they should never contain hardcoded URLs or raw network logic. By isolating all Axios HTTP calls inside CustomerService.js, the rest of the application remains clean and decoupled. Every function in this file returns a JavaScript **Promise**. When a component calls one of these functions, Axios sends the network request asynchronously, converts the incoming JSON response into a native JavaScript object, and allows the calling component to chain `.then()` for successful operations and `.catch()` for network or server errors. If the backend URL or port ever changes, you only have to modify the single `API_URL` variable in this file rather than searching through multiple UI components.

2. File: CustomerForm.jsx

Path: src/components/CustomerForm.jsx • **Role:** Controlled Add & Edit Form Component

```
function CustomerForm({ customerToEdit, onSave, onCancel }) {

  const [cname, setCname] = useState("");
  const [productName, setProductName] = useState("");
  const [price, setPrice] = useState("");
  const [quantity, setQuantity] = useState("");

  const [message, setMessage] = useState("");
  const [error, setError] = useState("");

  useEffect(() => {
    if (customerToEdit) {
      setCname(customerToEdit.cname);
      setProductName(customerToEdit.productName);
      setPrice(customerToEdit.price);
      setQuantity(customerToEdit.quantity);
    }
  }, [customerToEdit]);

  const handleSubmit = (event) => {
    event.preventDefault();
    setMessage("");
    setError("");

    const customer = {
      cname: cname,
      productName: productName,
      price: Number(price),
      quantity: Number(quantity)
    };

    if (customerToEdit) {
      CustomerService.updateCustomer(customerToEdit.cid, customer)
        .then(response => {
          setMessage("Customer updated successfully!");
          clearForm();
          onSave();
        })
        .catch(error => setError("Unable to update customer."));
    } else {
      CustomerService.createCustomer(customer)
        .then(response => {
          setMessage("Customer created successfully!");
          clearForm();
          onSave();
        })
        .catch(error => {
          if (error.response) {
            setError("Create failed. Status: " + error.response.status);
          } else {
            setError("Cannot connect to Spring Boot.");
          }
        });
    }
  };
}
```

```

const clearForm = () => {
  setName(""); setProductName(""); setPrice(""); setQuantity("");
};

const handleCancel = () => {
  clearForm(); setMessage(""); setError(""); onCancel();
};

return (
  <div className="form-card">
    <h2>{customerToEdit ? "■ Update Customer" : "■ Add Customer"}</h2>
    {message && <div className="success-message">{message}</div>}
    {error && <div className="error-message">{error}</div>}
    <form onSubmit={handleSubmit}>
      <label>Customer Name</label>
      <input type="text" placeholder="Enter customer name" value={cname}
        onChange={(e) => setName(e.target.value)} required />

      <label>Product Name</label>
      <input type="text" placeholder="Enter product name" value={productName}
        onChange={(e) => setProductName(e.target.value)} required />

      <label>Price</label>
      <input type="number" placeholder="Enter price" value={price}
        onChange={(e) => setPrice(e.target.value)} required />

      <label>Quantity</label>
      <input type="number" placeholder="Enter quantity" value={quantity}
        onChange={(e) => setQuantity(e.target.value)} required />

      <div className="form-buttons">
        <button type="submit" className="save-btn">
          {customerToEdit ? "Update Customer" : "Save Customer"}
        </button>
        <button type="button" className="clear-btn" onClick={handleCancel}>
          Clear
        </button>
      </div>
    </form>
  </div>
);
}
export default CustomerForm;

```

Line-by-Line Breakdown:

- **CustomerForm({ customerToEdit, onSave, onCancel })**: Declares the functional component and destructures three incoming props from App.jsx.
- **const [cname...], [productName...], [price...], [quantity...]**: Four state variables holding the live input texts for Customer Name, Product Name, Price, and Quantity.
- **const [message...], [error...]**: State variables tracking positive confirmation messages or error notification banners.
- **useEffect(..., [customerToEdit])**: Monitors the customerToEdit prop. When non-null (Edit mode), it pre-fills the four input states with that customer's existing data.
- **handleSubmit(event)**: Form submission handler. Calls event.preventDefault() to stop browser reload, casts numeric inputs via Number(), and evaluates the active mode.
- **if (customerToEdit) ... else ...**: Submits to updateCustomer or createCustomer, shows confirmation alerts, wipes inputs via clearForm(), and executes onSave().

- **clearForm() & handleCancel():** Resets all four inputs to blank strings and invokes onCancel() to tell App.jsx to exit edit mode.
- **Controlled Inputs (<input value={...} onChange={...}>):** Implements React's Controlled Component pattern—React state drives the input value, and user keystrokes update state via onChange.
- **Dynamic Header & Buttons:** Toggles labels between 'Update Customer' and 'Save Customer' according to mode.

In-Depth Explanation — CustomerForm.jsx

CustomerForm.jsx is an intelligent, dual-mode component engineered to serve both creation and editing operations without code duplication. It utilizes React's **Controlled Components**, meaning that every keystroke in an input field directly updates the component's internal state through React's useState hook, making the React state the single source of truth for all form data. The component's reactivity is driven by the useEffect hook: when the user clicks the 'Edit' button on any row in the customer table, App.jsx passes that customer down through the customerToEdit prop. The useEffect hook immediately detects this prop change and pre-fills all input fields with that customer's existing attributes, while dynamically switching the title and submit button to 'Update Customer'. When submitted, the handler checks whether customerToEdit exists to branch between a POST or PUT request via CustomerService. After completion, the form resets itself and notifies the parent via the onSave callback, ensuring a synchronized and responsive user experience.

3. File: CustomerList.jsx

Path: src/components/CustomerList.jsx • **Role:** Presentational Data Table Component

```

function CustomerList({ customers, onEdit, onDelete, onRefresh }) {
  return (
    <div className="list-card">
      <div className="list-header">
        <h2>■ All Customers</h2>
        <button className="refresh-btn" onClick={onRefresh}>
          ■ Refresh
        </button>
      </div>

      <table>
        <thead>
          <tr>
            <th>ID</th>
            <th>Customer Name</th>
            <th>Product Name</th>
            <th>Price</th>
            <th>Quantity</th>
            <th>Actions</th>
          </tr>
        </thead>

        <tbody>
          {customers.length === 0 ? (
            <tr>
              <td colspan="6">No customers found</td>
            </tr>
          ) : (
            customers.map(customer => (
              <tr key={customer.cid}>
                <td>{customer.cid}</td>
                <td>{customer.cname}</td>
                <td>{customer.productName}</td>
                <td>■{customer.price}</td>
                <td>{customer.quantity}</td>
                <td>
                  <button className="edit-btn" onClick={() => onEdit(customer)}>
                    =■ Edit
                  </button>
                  <button className="delete-btn" onClick={() => onDelete(customer.cid)}>
                    ■■ Delete
                  </button>
                </td>
              </tr>
            ))
          )}
        </tbody>
      </table>
    </div>
  );
}
export default CustomerList;

```

Line-by-Line Breakdown:

- **CustomerList({ customers, onEdit, onDelete, onRefresh })**: Declares the functional presentation component receiving the data array and three action callbacks.
- **<button className="refresh-btn" onClick={onRefresh}>**: Header button triggering the parent's refresh callback to re-query MySQL.

- **<table> <thead> ... </thead>**: Standard HTML table markup defining the headers for ID, Customer Name, Product Name, Price, Quantity, and Actions.
- **{customers.length === 0 ? ... : ...}**: Conditional ternary check rendering a single 'No customers found' row across all six columns (colSpan="6") if the list is empty.
- **customers.map(customer => ...)**: Iterates over the customer array, returning a distinct <tr> row for each record.
- **key={customer.cid}**: Unique key assigned to each row, enabling React's reconciliation engine to efficiently target DOM updates.
- **onClick={() => onEdit(customer)}**: Inline arrow function forwarding the complete customer object back to App.jsx on click.
- **onClick={() => onDelete(customer.cid)}**: Inline arrow function forwarding the customer ID to App.jsx to initiate deletion.

In-Depth Explanation — CustomerList.jsx

CustomerList.jsx is a **Presentational Component** (stateless). It contains no internal state and is entirely driven by the props passed down from App.jsx. Its sole responsibility is to map the customer dataset into a clean tabular presentation and bubble user events (edit, delete, refresh) up to the parent. When rendering the table, it uses JavaScript's .map() method to transform an array of customer data objects into JSX table rows. A key technical requirement here is the key={customer.cid} attribute on each <tr>: React's Virtual DOM algorithm relies on unique keys to identify which specific rows have changed, been added, or been removed, avoiding costly full-table re-renders. Furthermore, the Edit and Delete buttons utilize inline arrow functions (e.g., () => onEdit(customer)); this guarantees that the callback functions are only executed when the user physically clicks the button, rather than firing automatically during the initial render pass.

4. File: App.jsx

Path: src/App.jsx • **Role:** Central State Store & Application Orchestrator

```
function App() {
  const [customers, setCustomers] = useState([]);
  const [customerToEdit, setCustomerToEdit] = useState(null);
  const [searchId, setSearchId] = useState("");
  const [searchedCustomer, setSearchCustomer] = useState(null);
  const [searchError, setSearchError] = useState("");

  const getCustomers = () => {
    CustomerService.getAllCustomers()
      .then(response => setCustomers(response.data))
      .catch(error => console.log(error));
  };

  useEffect(() => {
    getCustomers();
  }, []);

  const getCustomerById = () => {
    if (!searchId) {
      setSearchError("Please enter Customer ID.");
      setSearchCustomer(null);
      return;
    }
    CustomerService.getCustomerById(searchId)
      .then(response => {
        setSearchCustomer(response.data);
        setSearchError("");
      })
      .catch(error => {
        setSearchCustomer(null);
        setSearchError("Customer not found.");
      });
  };

  const deleteCustomer = (cid) => {
    const confirmDelete = window.confirm(
      "Are you sure you want to delete Customer ID " + cid + "?"
    );
    if (!confirmDelete) return;

    CustomerService.deleteCustomer(cid)
      .then(response => {
        alert("Customer deleted successfully!");
        getCustomers();
      })
      .catch(error => alert("Unable to delete customer."));
  };

  const editCustomer = (customer) => {
    setCustomerToEdit(customer);
    window.scrollTo({ top: 0, behavior: "smooth" });
  };

  const handleSaved = () => { setCustomerToEdit(null); getCustomers(); };
  const handleCancel = () => { setCustomerToEdit(null); };
}
```

```

return (
  <div className="app-container">
    <header>
      <h1>■ Customer Management System</h1>
      <p>React + Spring Boot + MySQL</p>
    </header>

    <main>
      <div className="top-section">
        <CustomerForm
          customerToEdit={customerToEdit}
          onSave={handleSaved}
          onCancel={handleCancel}
        />

        <div className="search-card">
          <h2>■ Get Customer By ID</h2>
          <label>Enter Customer ID</label>
          <div className="search-box">
            <input
              type="number" placeholder="Enter ID" value={searchId}
              onChange={(e) => setSearchId(e.target.value)}
            />
            <button className="search-btn" onClick={getCustomerById}>
              ■ Get Customer
            </button>
          </div>
          {searchError && <p className="error-message">{searchError}</p>}
          {searchedCustomer && (
            <div className="customer-details">
              <h3>Customer Details</h3>
              <p><strong>ID:</strong> {searchedCustomer.cid}</p>
              <p><strong>Name:</strong> {searchedCustomer.cname}</p>
              <p><strong>Product:</strong> {searchedCustomer.productName}</p>
              <p><strong>Price:</strong> ■{searchedCustomer.price}</p>
              <p><strong>Quantity:</strong> {searchedCustomer.quantity}</p>
            </div>
          )}
        </div>

        <CustomerList
          customers={customers}
          onEdit={editCustomer}
          onDelete={deleteCustomer}
          onRefresh={getCustomers}
        />
      </main>

      <footer>Customer Management System • Built with React + Spring Boot + MySQL</footer>
    </div>
  );
}
export default App;

```

Line-by-Line Breakdown:

- **const [customers...]:** Master state holding the complete array of customer records fetched from the database.
- **const [customerToEdit...]:** Holds null during create mode, or the selected customer object during edit mode.
- **const [searchId...], [searchedCustomer...], [searchError...]:** State trio powering the single-record search feature.
- **getCustomers():** Calls `CustomerService.getAllCustomers()` and updates `customers` with `response.data`.

- **useEffect(..., [])**: Configured with an empty dependency array [], ensuring `getCustomers()` executes once on initial page load.
- **getCustomerById()**: Validates input, queries the API for that customer ID, and updates either `searchedCustomer` or `searchError`.
- **deleteCustomer(cid)**: Displays `window.confirm()` dialog; upon approval, calls `DELETE` and re-runs `getCustomers()` to refresh the table.
- **editCustomer(customer)**: Sets `customerToEdit` and smoothly scrolls the viewport to the top (`window.scrollTo`).
- **handleSaved() & handleCancel()**: Resets `customerToEdit` back to null and refreshes the data table after successful modifications.
- **Layout Assembly**: Organizes the screen into a top CSS Grid section (Form on left, Search Card on right) and a full-width Table section below.

In-Depth Explanation — App.jsx

`App.jsx` is the **Root Orchestrator** and central state store of the frontend application. It applies the fundamental React pattern known as **Lifting State Up**: because both `CustomerForm` and `CustomerList` need access to the customer data and need to know which record is being edited, that state cannot reside inside either child component—it must live in their closest common ancestor, `App.jsx`. When the application first mounts, the `useEffect` hook automatically fires `getCustomers()` to populate the table. Whenever a user interacts with a child component (for example, clicking 'Delete' or 'Edit' inside `CustomerList`), the child component calls a callback function provided by `App.jsx`. `App.jsx` handles the business logic, interacts with `CustomerService`, updates its state variables, and React automatically re-renders the relevant parts of the UI. This unidirectional data flow (state flowing down via props, events flowing up via callbacks) ensures predictable state management across the entire application.

5. Files: main.jsx & index.html

Role: DOM Mounting & Application Bootstrap

```
// src/main.jsx
ReactDOM.createRoot(
  document.getElementById("root")
).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

// index.html (Snippet)
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

Line-by-Line Breakdown:

- **document.getElementById("root")**: Locates the empty host `<div id="root">` defined inside `index.html`.
- **ReactDOM.createRoot(...)**: Initializes React 19's concurrent rendering root on that physical DOM element.
- **<React.StrictMode>**: Development wrapper that invokes lifecycle checks and effects twice to expose potential memory leaks and side-effects.
- **<App />**: Mounts the root application component tree into the browser DOM.

- `<script type="module" src="/src/main.jsx">`: Directs the browser to execute main.jsx as an ES module supported natively by Vite.

In-Depth Explanation — main.jsx & index.html

main.jsx is the **Entry Point** and mounting bridge of the frontend application. In a React Single Page Application, the browser initially loads an almost empty HTML file (index.html). main.jsx is the first JavaScript file executed by the browser: it targets the root div element on the page using standard DOM query methods and hands complete rendering control over to React using ReactDOM.createRoot(). Wrapping the top-level `<App />` component in `<React.StrictMode>` activates additional runtime checks during local development, such as verifying that side-effects are properly cleaned up and warning about deprecated APIs. This file is executed once when the browser loads the page, establishing the Virtual DOM tree that powers the entire interactive user experience.